

Vayu Firmware — Flight Control & the Vehicle Control-Output Path

Implementation reference, with emphasis on the rate-loop → motor pipeline

Vayu firmware · `src/control`, `src/actuator`

June 2026

Abstract

How the Vayu firmware turns pilot stick input and sensor feedback into four motor commands: the cascaded angle→rate controller, the PID core, and — in detail — the **control-output pipeline** (rate-PID outputs → authority ramp → geometry-derived mix → anti-saturation → idle floor → arming gate → ESC pulse). Reflects the implementation as of the current tree.

Contents

1. Scope	1
2. Architecture, threading, and data flow	1
3. The cascade and the PID core	2
4. The vehicle control-output pipeline (<i>focus</i>)	2
4.0 PID reset on arm	2
4.1 Throttle clamp	3
4.2 Integrator gate (anti-windup on the ground)	3
4.3 Rate PID	3
4.4 Authority ramp	3
4.5 Geometry-derived mix	3
4.6 Anti-saturation (preserve throttle, sacrifice attitude)	3
4.7 Final clip + NaN guard	4
4.8 Idle floor	4
4.9 Arming gate	4
4.10 Motor task → ESC → output	4
5. Flight modes & failsafe at the output	4
6. Runtime tunability & persistence	4
7. Constant reference	5
8. Control-output telemetry	5
9. Design decisions worth remembering	5
10. File map	6

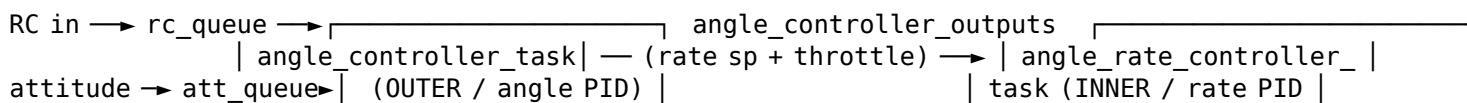
1. Scope

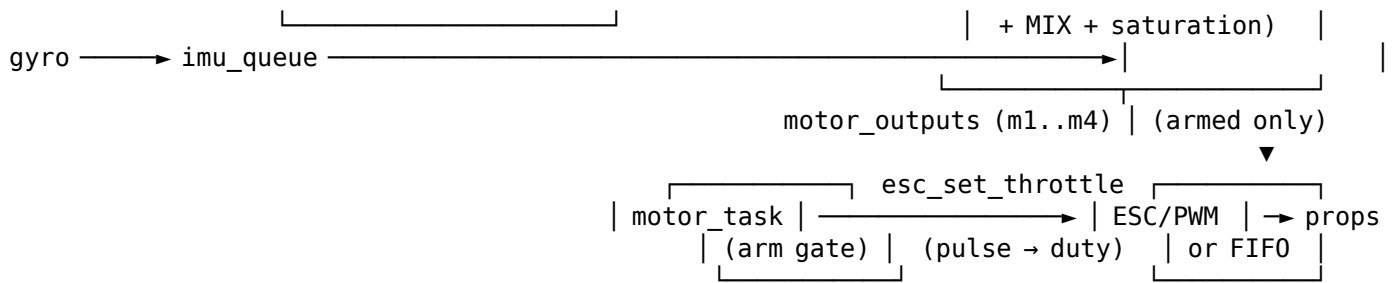
This report documents the **flight-control implementation** of the Vayu firmware, with the bulk of the detail on the **vehicle control output** — every stage between “the rate PID produced a correction” and “the ESC sees a pulse width.” That output path is where most of the safety logic, the failure modes, and the non-obvious engineering live.

All control code is **shared between hardware and SITL** — only the HAL is swapped. The same `src/control/*` and `src/actuator/*` run in the simulator; `tools/sim_host` provides the PWM/IMU/RC shims.

2. Architecture, threading, and data flow

Four cooperating tasks (FreeRTOS tasks on hardware; detached pthreads in SITL, created in `tools/sim_host/src/host_lifecycle.c`), communicating through single-producer/single-consumer queues:





Loop rates. The inner rate loop is **event-driven by IMU samples** (BMX160 ODR BMX_GYR_ODR = 1600 Hz), bounded by a 5 ms wait (RATE_LOOP_MAX_PERIOD_MS) so it runs at ≥ 200 Hz even if samples stall; in SITL the IMU pace sets it (~ 200 Hz). The outer loop is RC/attitude-paced (~ 1 kHz, $v_delay(2)$). dt for each loop is measured from the cycle counter (SYS_CLOCK_FREQ = 84 MHz), not assumed.

3. The cascade and the PID core

Outer (angle) loop — src/control/angle_controller.c. P-only (K_p only; $K_i = K_d = 0$). Stick $\rightarrow$ target angle ($\pm \text{DEFAULT_ANGLE_TARGET_MAX}$, default 100° , through a cubic expo that is **clamped to ± 1 before cubing** so a glitch/startup RC value can't produce a runaway setpoint). Output: a **body-rate setpoint** per axis, plus the throttle pass-through.

Inner (rate) loop — src/control/angle_rate_controller.c. Full PID per axis, operating on the gyro (deadbanded at $\text{PID_GYRO_DEADBAND} = 0.1^\circ/\text{s}$, then an optional per-axis **gyro low-pass**). Output: a normalized correction outputs[roll, pitch, yaw] $\in [-1, 1]$.

PID update (src/control/pid.c), error $e = \text{sp} - \text{meas}$:

$$\begin{aligned}
 P &= K_p e \\
 I &= \text{clamp}(I + K_i e \Delta t, \pm i_{\max}) \\
 D_{\text{raw}} &= -K_d \frac{\text{meas} - \text{meas}_{\text{prev}}}{\Delta t} \quad (\text{derivative on measurement — no setpoint kick}) \\
 D &= \text{clamp}\left(\text{LPF}(D_{\text{raw}}), \pm d_{\max}\right), \quad \alpha = \frac{\Delta t}{\Delta t + \tau_{\text{lpf}}} \\
 \text{out} &= \text{clamp}(P + I + D + K_{ff} \dot{s}_p, \text{out}_{\min}, \text{out}_{\max})
 \end{aligned}$$

Conditional integration (anti-windup): the integrator is only *committed* if $P + I$ alone is within the output limits — if the proportional+integral term is already saturating, I is frozen rather than wound further. $dt \leq 1e-6$ returns 0 (divide-by-zero guard). K_{ff} is wired but unused ($\dot{s}_p = 0$).

4. The vehicle control-output pipeline (*focus*)

This is the ordered sequence in angle_rate_controller_task that converts the three PID corrections + throttle into four ESC commands. Each stage exists for a specific failure it prevents.

4.0 PID reset on arm

On every STANDBY $\rightarrow$ ARMED transition, all three rate PIDs are reset (integrator, prev_meas, D-filter) and the gyro-LPF state cleared, so no windup or stale derivative carries across an arm cycle.

4.1 Throttle clamp

target_throttle is clamped to $[0, 1]$. A sub-1000 μs “idle” from an imperfectly-trimmed TX would otherwise give a slightly negative throttle, which makes the §4.5 anti-saturation divide by zero $\rightarrow$ NaN $\rightarrow$ “nan” down the PWM FIFO $\rightarrow$ exploded physics. The clamp is the cheap guard.

4.2 Integrator gate (anti-windup on the ground)

If target_throttle < RATE_PID_INTEGRATE_THROTTLE (0.3), every integrator is held at 0. On the ground the airframe can’t rotate, so a standing rate error would otherwise wind the I-term up and slam a motor the instant throttle crosses hover.

4.3 Rate PID

outputs[i] = v_pid_update(pid[i], target_rates[i], current_rates[i], ...) for roll/pitch/yaw — the §3 PID, producing corrections in $[-1, 1]$.

4.4 Authority ramp

The PID corrections are gated by throttle:

$$\text{outputs} *= \begin{cases} 0 & \text{thr} < \text{MIN_ARMED_THROTTLE} (0.1) \\ \frac{\text{thr} - \text{MIN_ARMED_THROTTLE}}{\text{PID_FULL_AUTHORITY_THROTTLE} - \text{MIN_ARMED_THROTTLE}} & \text{MIN_ARMED} \leq \text{thr} < \text{FULL_AUTH} \\ 1 & \text{thr} \geq \text{PID_FULL_AUTHORITY_THROTTLE} (0.3) \end{cases}$$

Rationale: below near-hover the props can’t actually rotate the airframe; a PID correction there only fights the ground contact, and that couples with the Mahony estimator into a positive-feedback tip-over (wobble $\rightarrow$ big rate demand $\rightarrow$ motor deflect $\rightarrow$ tips further $\rightarrow$ filter sees more $\rightarrow$...). Gating to ~hover breaks the loop.

4.5 Geometry-derived mix

Each motor is the throttle plus signed PID contributions, with the **per-motor signs derived from the airframe geometry** (position + spin), not a hardcoded numbering — so any quad layout is consistent with the physics:

$$\text{out}_i = \text{thr} + \text{roll} \cdot (-\text{sign } y_i) + \text{pitch} \cdot (\text{sign } x_i) + \text{yaw} \cdot \text{spin}_i$$

Defaults reproduce the legacy X-quad (FR=M1, RR=M2, RL=M3, FL=M4). The signs are set at boot/runtime via angle_rate_controller_set_motor_geometry() / CMD_SET_MOTOR_GEOMETRY, from the *same* geometry pushed to the sim — see docs/analysis/autotune-methodology.md §8 for why a mismatch here flips the craft.

4.6 Anti-saturation (preserve throttle, sacrifice attitude)

If any motor falls outside $[0, 1]$, **all PID differentials are scaled by one factor** $s \leq 1$ so the worst motor sits exactly at a limit while the pilot’s throttle is preserved:

$$s = \min\left(1, \underbrace{\frac{\text{thr}}{\text{thr} - m_{\min}}}_{\text{if } m_{\min} < 0}, \underbrace{\frac{1 - \text{thr}}{m_{\max} - \text{thr}}}_{\text{if } m_{\max} > 1}\right), \quad m_i \leftarrow \text{thr} + s(m_i - \text{thr})$$

This deliberately replaces the older “shift everything up by $|m_{\min}|$ ” trick, which silently *added un-commanded thrust* (a hard roll demand at low throttle made the craft shoot up and roll). Here attitude authority degrades when limits bite, but commanded thrust is exact.

4.7 Final clip + NaN guard

Motors clipped to $[0, 1]$; then an explicit `isnan()`→0 per motor, because the ordered comparisons (<0 , >1) are *false* for NaN and would let one slip through into the ESC.

4.8 Idle floor

While armed, each motor is held at $\geq \text{MOTOR_IDLE_FLOOR}$ (0.005). Mirrors a real ESC’s `MOTOR_STOP=false` (props idle so the next command doesn’t cold-start), and in SITL distinguishes *armed-at-idle* from *disarmed* for the physics bridge’s gravity logic. `motor_task` still zeroes outputs when not ARMED, so the floor only applies in flight.

4.9 Arming gate

`motor_set_outputs()` feeds the motor FIFO **only while** `SYSTEM_STATE_ARMED`. The FIFO is overwrite-policy; without this gate, a pre-arm asymmetric mix (PID acting on gyro noise while disarmed) sits in the slot and is served as a **step kick** the instant the state flips to ARMED — flipping the craft before throttle is even touched. Gating eliminates that stale slot.

4.10 Motor task → ESC → output

`motor_task` (`src/actuator/motor.c`) reads the FIFO and, if **not** ARMED, forces all four to 0 (defence-in-depth with §4.9), then calls `esc_set_throttle` per motor. `esc_set_throttle` (`src/actuator/esc.c`) maps the normalized command to a servo pulse and then a timer duty:

$$\text{pulse} = \text{min_pulse} + \text{cmd} \cdot (\text{max_pulse} - \text{min_pulse}), \quad \text{duty} = \frac{\text{pulse}}{1000 / f_{\text{esc}}}$$

- **Hardware:** `hal_pwm_set_duty_cycle` drives the timer → ESC PWM/DShot.
- **SITL:** `tools/sim_host/src/host_navhal.c` clamps duty to $[0, 1]$ and writes the four values to the `/tmp/vsim_pwm` FIFO (latest-wins) for `vsim_d` to integrate.

5. Flight modes & failsafe at the output

- **Stabilize (angle) mode** — default. Stick → angle → rate → output, with the $\pm 70^\circ$ **bank-angle cutoff** (`MAX_ANGLE_CUTOFF`): exceeding it trips `SYSTEM_STATE_FAILSAFE`. Yaw is excluded from the cutoff (free to spin; also the mag-less sim yaw drifts).
- **Acro (rate) mode** — RC **channel 6** high. The angle PID is bypassed; sticks command body rate directly ($\pm \text{DEFAULT_ACRO_RATE_MAX}$, 200 °/s), and the cutoff failsafe is suppressed so the craft can flip/roll continuously. Everything from §4.4 onward is identical.
- **Disarm / non-ARMED** — `motor_task` forces all outputs to 0; `motor_set_outputs` isn’t even fed (§4.9). Two independent paths to “motors off.”

6. Runtime tunability & persistence

The output path is reconfigurable in-flight over the command link, each persisted to `0:pid.bin` (store format PID2) and reloaded at boot by `pid_config_init()`:

Command	Payload	Effect
CMD_SET_PID (0x000A)	controller, axis, Kp, Ki, Kd, Kff	live rate/angle gains
CMD_SET_GYRO_LPF (0x000B)	axis, τ	rate-loop input low-pass
CMD_SET_MOTOR_GEOMETRY (0x000C)	x[4], y[4], spin[4]	mixer signs (§4.5)

(On real hardware these persist to SD; in SITL the host VFS is disk-backed so a tune survives restart.)

7. Constant reference

Constant	Value	Role
SYS_CLOCK_FREQ	84 MHz	dt timebase
BMX_GYR_ODR	1600 Hz	gyro sample rate → inner-loop pace
RATE_LOOP_MAX_PERIOD_MS	5 ms	inner-loop watchdog (≥ 200 Hz)
PID_GYRO_DEADBAND	0.1 °/s	gyro deadband
PID_RC_DEADBAND	10 μ s	stick centre deadband
RATE_PID_INTEGRATE_THROTTLE	0.30	integrator gate (§4.2)
MIN_ARMED_THROTTLE	0.10	authority-ramp floor (§4.4)
PID_FULL_AUTHORITY_THROTTLE	0.45 HW / 0.30 sim	authority-ramp ceiling
MOTOR_IDLE_FLOOR	0.005	armed idle thrust (§4.8)
MAX_ANGLE_CUTOFF	70°	stabilize-mode failsafe
rate i_max / d_max / d_lpf_rc	0.2 / 0.25 / 0.3	per-axis PID limits + D-LPF τ

(Rate/angle gains themselves are seeds in include/variables.h, overridden by the persisted tune; the VAYU_SIM build ships gentler seeds.)

8. Control-output telemetry

Every inner-loop iteration emits control_telemetry_t — 18 floats: angle setpoint/measured $\times 3$, rate setpoint/measured $\times 3$, the three PID outputs, throttle, and the two loop dts — as PACKET_TYPE_SYSTEM_STATUS origin 0x05 (~18 Hz). This is the single richest observability point for the whole output path (and what the autotuner and HUD consume).

9. Design decisions worth remembering

- **Throttle is sacred; attitude is negotiable.** Anti-saturation (§4.6) and the authority ramp (§4.4) both preserve commanded thrust and give up attitude authority under stress — the opposite of the old “shift-up” mix that injected uncommanded lift.
- **Two independent “motors off” paths** (arm gate + motor_task zeroing) and two NaN guards (throttle clamp + isnan→0) — defence-in-depth on the actuator.
- **Geometry-derived mix** keeps the firmware mixer and the physics/airframe in lockstep; a layout mismatch is the classic “stable inverted” bug.

- **Everything is gated on real arm state and real throttle**, not on the pilot's *intent* — which is why the ground-feedback tip-over and the pre-arm step-kick were both fixed at the output stage, not in the PID.
-

10. File map

File	Responsibility
src/control/angle_controller.c	outer loop, stick→angle, expo, acro branch, cutoff
src/control/angle_rate_controller.c	inner loop, gyro LPF, \$4 output pipeline , mix
src/control/pid.c	PID core (\$3)
src/control/pid_config.c	gain/LPF persistence, CMD_SET_* apply
src/actuator/motor.c	motor task, arm-state zeroing
src/actuator/esc.c	throttle → pulse → duty
tools/sim_host/src/host_navhal.c	SITL PWM → /tmp/vsim_pwm
include/variables.h	constants + gain seeds